Study Companion: Introduction to Data Structures

Computer Science Fundamentals

1 The big idea: Organizing Information

How we arrange data changes how fast we can find or change it.

When we write programs, we deal with data. But data doesn't just float around in the computer's memory randomly. We have to decide *how* to organize it. This organization is called a **data structure**. Choosing the right data structure can make a program thousands of times faster.

Intuition

A data structure is like a physical storage system. If you throw all your tools into a giant bucket, finding a screwdriver takes a long time. If you hang them on a pegboard, finding one is instant. The tools are the same, but the structure changes how you work with them.

Everyday picture

Picture a stack of plates in a cafeteria. You can only easily take the top plate, and if you add a plate, it goes on top. Now picture a line of people waiting for a bus. The first person in line is the first one to get on. The stack and the line are two different ways to organize things, each with its own rules.

2 Arrays: The Bookshelf

A row of slots in memory, numbered in order.

An **array** is the simplest data structure. It is a continuous block of memory divided into equal-sized slots. Each slot holds one item.

We find items in an array using an **index**. In most languages, we start counting at zero. So the first item is at index 0, the second at index 1, and so on.

Worked example: Finding a book

Suppose we have an array of 5 book titles.

Step 1. We create an array `books = ["Dune", "1984", "Foundation", "Emma", "Beloved"]`.

Step 2. We want the third book. Since we start at 0, the third book is at index 2.

Step 3. We look up `books[2]`. The computer instantly finds "Foundation".

Watch out

Arrays are fixed in size. When you create an array of 5 items, the computer carves out exactly that much space. If you want to add a 6th item later, you cannot simply tack it on. Instead, you must create a new, larger array and copy everything over.



Figure 1: A simple visual placeholder for array.

3 Linked Lists: The Scavenger Hunt

Items scattered in memory, holding hands to stay connected.

If arrays are rigid, **linked lists** are flexible. Instead of one continuous block, a linked list is made of separate **nodes**. Each node holds two things: the actual data, and a pointer (a map) to the next node.

Intuition

Imagine a scavenger hunt. The first clue doesn't contain the prize, but it tells you exactly where the second clue is hidden. The second clue points to the third, and so on. You don't need a huge open field to set this up; the clues can be hidden anywhere.

Worked example: Following the links

Let's build a linked list with three numbers: 10, 20, 30.

Step 1. Create the first node: holds `10`, points to the second node.

Step 2. Create the second node: holds `20`, points to the third node.

Step 3. Create the third node: holds `30`, points to nothing (`null`).

Step 4. To find `30`, we *must* start at the first node, follow its pointer to `20`, and follow that pointer to `30`.

Key takeaway

Arrays are fast for looking things up if you know the index, but slow to grow. Linked lists are easy to grow and shrink, but slow to search because you have to walk the list one by one.

Where it's used. When using the "undo" button in a text editor, you often walk backward through a linked list of your past actions.

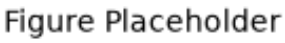


Figure 2: A simple visual placeholder for linked list.

Glossary & symbols

Key terms.

data structure	A specific way to organize and store data in a computer.
array	A continuous block of memory with numbered slots.
linked list	A chain of nodes, where each node points to the next.